Habiba Abdullah Said Hammad
2203167
KBAI- Assignment 1

MediAssist: A medical RAG system

1. Requirements and Knowledge Base Curation

After deciding on the topic child obesity, 13 papers were collected from PubMed Open Access and WHO. I made sure that they were in pdf format according.

Papers are found in the files folder in the drive. They are of medium length and include topics such as

- Etiology and risk factors
- Diagnosis and differential diagnosis
- Treatment and management strategies
- Long-term complications and comorbidities
- Policy, screening, and prevention

The paper's sources include:

- Academic journals
- Systematic reviews and meta-analyses
- Position papers from medical organizations

Habiba Abdullah Said Hammad
2203167
KBAI- Assignment 1

2. Ingestion pipeline and Embedding Module

I then created an ingest.py file which loads the files and then uses RecursiveCharacterTextSplitter in order to create chunks of the documents.

```python
import os
from langchain_community.document_loaders import PyPDFLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_huggingface import HuggingFaceEmbeddings
from langchain_community.vectorstores import FAISS

def load_and_chunk_documents(file_paths):
    documents = []

    for path in file_paths:
        if path.endswith(".pdf") and os.path.exists(path):
            loader = PyPDFLoader(path)
            documents.extend(loader.load())
            print(f"Loaded PDF: {path}")
        else:
            print(f"Skipped or not found: {path}")

    text_splitter = RecursiveCharacterTextSplitter(
        chunk_size=512,
        chunk_overlap=64
    )

    chunks = text_splitter.split_documents(documents)
    print(f"\nSuccess! Generated {len(chunks)} chunks.")
    return chunks

```

(Note that langchain library was used heavily in this assignment)

After that embeddings were created using Bio_ClinicalBERT model (embeddings were also normalized to ensure semantic similarity) and the FAISS database (which is an open-source library from META which is used for similarity search).

```python
if __name__ == "__main__":

    my_files = ["Files/Obesity1.pdf", "Files/Obesity2.pdf", "Files/Obesity3.pdf", "Files/Ol
    doc_chunks = load_and_chunk_documents(my_files)
    if doc_chunks:
        print("\nInitializing BioBERT Embeddings (This will download the model on the firs
        embeddings = HuggingFaceEmbeddings(model_name="emilyalsentzer/Bio_ClinicalBERT", e
        print("Building the FAISS database...")
        vectorstore = FAISS.from_documents(doc_chunks, embeddings)
        vectorstore.save_local("faiss_index")
        print("Success! Database saved locally in the 'faiss_index' folder.")
```

Habiba Abdullah Said Hammad
2203167
KBAI- Assignment 1

3. Retrieval Engine and Vector Store

Here the embedding model and FAISS database are loaded. THE FAISS is put into a vector store which is then used for the retriever. Here it selects the top 5 chunks per query with a minimum similarity of 0.65

```python
from langchain_community.vectorstores import FAISS
from langchain_huggingface import HuggingFaceEmbeddings

print("Loading embedding model...")
embeddings = HuggingFaceEmbeddings(model_name="emilyalsentzer/Bio_ClinicalBERT",encode_kwa

print("Loading FAISS database...")

vectorstore = FAISS.load_local(
    "faiss_index",
    embeddings,
    allow_dangerous_deserialization=True
)

retriever = vectorstore.as_retriever(
    search_type="similarity_score_threshold",
    search_kwargs={
        "k": 5,
        "score_threshold": 0.65
    }
)
```

4.LLM integration and prompt engineering

For this assignment I decided to go with groq LLM as it is fast and open source. After going into the website to generate an API key and putting it in a separate file (.env file) for privacy concerns, I loaded the llama-3.3-70b-versatile model and then added the prompt which followed the assignment's requirements.

Habiba Abdullah Said Hammad
2203167
KBAI- Assignment 1

```python
from dotenv import load_dotenv
from retriever import retriever,vectorstore

load_dotenv()

GROQ = os.getenv("GROQ_API_KEY")

os.environ["GROQ_API_KEY"] = GROQ
llm = ChatGroq(model_name="llama-3.3-70b-versatile", temperature=0)

system_prompt = (
    "You are MediAssist, a medical information assistant.\n"
    "Use the following pieces of retrieved context to answer the user's question.\n"
    "If you don't know the answer based on the context, say that you don't know. Do not hallucinate
    "CRITICAL: You MUST cite the source document and page number for every piece of information used
    "CRITICAL: At the very end of your response, you MUST append this exact text:\n"
    "'MEDICAL DISCLAIMER: This information is for educational purposes only and should not replace
    "Context:\n{context}"
)

prompt = ChatPromptTemplate.from_messages([
    ("system", system_prompt),
    ("human", "{input}"),
])

question_answer_chain = create_stuff_documents_chain(llm, prompt)
rag_chain = create_retrieval_chain(retriever, question_answer_chain)
```

Below is the full prompt:

"You are MediAssist, a medical information assistant.\n"

"Use the following pieces of retrieved context to answer the user's question.\n"

"If you don't know the answer based on the context, say that you don't know. Do not hallucinate.\n"

"CRITICAL: You MUST cite the source document and page number for every piece of information used (e.g., [Files/Obesity1.pdf, Page 3]).\n"

"CRITICAL: At the very end of your response, you MUST append this exact text:\n"

"'MEDICAL DISCLAIMER: This information is for educational purposes only and should not replace professional medical advice.'\n\n"

"Context:\n{context}"

Habiba Abdullah Said Hammad
2203167
KBAI- Assignment 1

It is also worth mentioning that the KB update feature was made by creating a file called kb_admin which contained 3 functions:get_database, add_documents _to_kb and remove_documents_from_kb

```python
import os
from langchain_community.vectorstores import FAISS
from langchain_huggingface import HuggingFaceEmbeddings
from ingest import load_and_chunk_documents
from retriever import embeddings


def get_database():
    """Loads the existing FAISS database."""
    if not os.path.exists("faiss_index"):
        print("Error: Knowledge Base not found. Run ingest.py first.")
        return None
    return FAISS.load_local("faiss_index", embeddings, allow_dangerous_deserialization=True)

def add_documents_to_kb(file_paths):
    """Adds new documents to the existing database without retraining."""
    vectorstore = get_database()
    if not vectorstore: return

    print(f"\nProcessing new documents: {file_paths}")
    new_chunks = load_and_chunk_documents(file_paths)

    if new_chunks:
        print("Adding chunks to the database...")
        vectorstore.add_documents(new_chunks)
        vectorstore.save_local("faiss_index")
        print("Success! Documents added and KB saved.")
    else:
        print("No valid documents found to add.")
```

```python
def remove_document_from_kb(filename):
    """Removes all chunks associated with a specific file without retraining."""
    vectorstore = get_database()
    if not vectorstore: return
    print(f"\nSearching database for chunks from '{filename}'...")
    docstore_dict = vectorstore.docstore._dict
    ids_to_delete = []
    for doc_id, document in docstore_dict.items():
        if document.metadata.get("source") == filename:
            ids_to_delete.append(doc_id)

    if ids_to_delete:
        print(f"Found {len(ids_to_delete)} chunks belonging to '{filename}'. Deleting...")
        vectorstore.delete(ids_to_delete)
        vectorstore.save_local("faiss_index")
        print("Success! Document removed and KB saved.")
    else:
        print(f"Error: No chunks found for '{filename}'. Check the exact file path.")
```

Habiba Abdullah Said Hammad
2203167
KBAI- Assignment 1

5. Front end and API

To finalize the assignment and make the model usable , FastAPI was used to turn the server on to host the web application which was made by streamlit.

```
load_dotenv()
GROQ = os.getenv("GROQ_API_KEY")
os.environ["GROQ_API_KEY"] = GROQ
class GraphState(TypedDict):
    question: str
    context: list
    answer: str
```

The graph of the RAG system has 2 nodes: retrieve node and generate node

```
def retrieve_node(state: GraphState):
    print("-> Node: Retrieving context from database...")
    docs = retriever.invoke(state["question"])
    return {"context": docs}
def generate_node(state: GraphState):
    print("-> Node: Generating answer with Llama 3.3...")
    prompt = ChatPromptTemplate.from_messages([
        ("system", system_prompt),
        ("human", "{question}"),
    ])
    formatted_context = "\n\n".join(
        [f"Source: {doc.metadata.get('source', 'Unknown')} | Page: {doc.metadata.get('page', 'Unknown
         for doc in state["context"]]
    )
    chain = prompt | llm
    response = chain.invoke({"question": state["question"], "context": formatted_context})
    return {"answer": response.content}
```

The final graph structure is shown below

Habiba Abdullah Said Hammad
2203167
KBAI- Assignment 1

```
workflow = StateGraph(GraphState)


workflow.add_node("retrieve", retrieve_node)
workflow.add_node("generate", generate_node)

workflow.add_edge(START, "retrieve")
workflow.add_edge("retrieve", "generate")
workflow.add_edge("generate", END)


app = workflow.compile()
```

[START]

|

▼

(retrieve) <-- Runs the `retrieve_node` function

|

▼

(generate) <-- Runs the `generate_node` function

|

▼

[END]

Finally the app.py file is used to create the front end of the RAG system

Habiba Abdullah Said Hammad
2203167
KBAI- Assignment 1

app.py > ...

```python
import streamlit as st
import requests
from kb_admin import add_documents_to_kb, remove_document_from_kb


st.set_page_config(page_title="MediAssist AI", page_icon="🩺", layout="wide")


CHAT_API_URL = "http://127.0.0.1:8000/chat"
ADD_DOC_API_URL = "http://127.0.0.1:8000/admin/add_doc"
REMOVE_DOC_API_URL = "http://127.0.0.1:8000/admin/remove_doc"

with st.sidebar:
    st.header("⚙️ Admin Panel")
    st.markdown("Manage Knowledge Base ")

    st.divider()
    st.subheader("Add Document")
    new_doc_path = st.text_input("File Path (e.g., Files/Obesity14.pdf)")
    if st.button(" Add to KB"):
        if new_doc_path:
            with st.spinner("Processing document..."):
                try:
                    response = requests.post(ADD_DOC_API_URL, json={"file_paths": [new_doc_path]})
                    if response.status_code == 200:
                        st.success(f"Added {new_doc_path}!")
                    else:
                        st.error(f"Error: {response.json().get('detail')}")
                except Exception as e:
                    st.error("Failed to connect to Admin API.")
        else:
            st.warning("Please enter a file path.")

    st.divider()

```

Habiba Abdullah Said Hammad
2203167
KBAI- Assignment 1

```python
    st.subheader("Remove Document")
    remove_doc_name = st.text_input("Exact Filename (e.g., Obesity1.pdf)")
    if st.button(" Remove from KB"):
        if remove_doc_name:
            with st.spinner("Deleting chunks..."):
                try:
                    response = requests.post(REMOVE_DOC_API_URL, json={"filename": remove_doc
                    if response.status_code == 200:
                        st.success(f"Removed {remove_doc_name}!")
                    else:
                        st.error(f"Error: {response.json().get('detail')}")
                except Exception as e:
                    st.error("Failed to connect to Admin API.")
        else:
            st.warning("Please enter a filename.")


st.title("🩺 MediAssist AI")
st.markdown("Your clinical information assistant. Ask a medical question below!")

if "messages" not in st.session_state:
    st.session_state.messages = []


for message in st.session_state.messages:
    with st.chat_message(message["role"]):
        st.markdown(message["content"])

if prompt := st.chat_input("Ask about childhood obesity treatments..."):


    with st.chat_message("user"):
        st.markdown(prompt)
    st.session_state.messages.append({"role": "user", "content": prompt})
```

```python
    with st.chat_message("assistant"):
        message_placeholder = st.empty()
        with st.spinner("Searching knowledge base..."):
            try:
                response = requests.post(CHAT_API_URL, json={"question": prompt})
                response.raise_for_status()
                answer = response.json().get("answer", "No answer found.")

                message_placeholder.markdown(answer)
                st.session_state.messages.append({"role": "assistant", "content": answer})

            except requests.exceptions.ConnectionError:
                st.error(" Cannot connect to the API. Make sure `python api.py` is running in another t
            except Exception as e:
                st.error(f"An error occurred: {e}")
```

Habiba Abdullah Said Hammad
2203167
KBAI- Assignment 1

And below is the api,py which includes the api data models and end poitns

```python
from fastapi import FastAPI, HTTPException
from pydantic import BaseModel
from typing import List
import uvicorn

from agent import app as langgraph_agent
from kb_admin import add_documents_to_kb, remove_document_from_kb

api = FastAPI(title="MediAssist API & Admin", version="2.0")

class ChatRequest(BaseModel):
    question: str

class ChatResponse(BaseModel):
    answer: str

class AddDocRequest(BaseModel):
    file_paths: List[str]

class RemoveDocRequest(BaseModel):
    filename: str
```

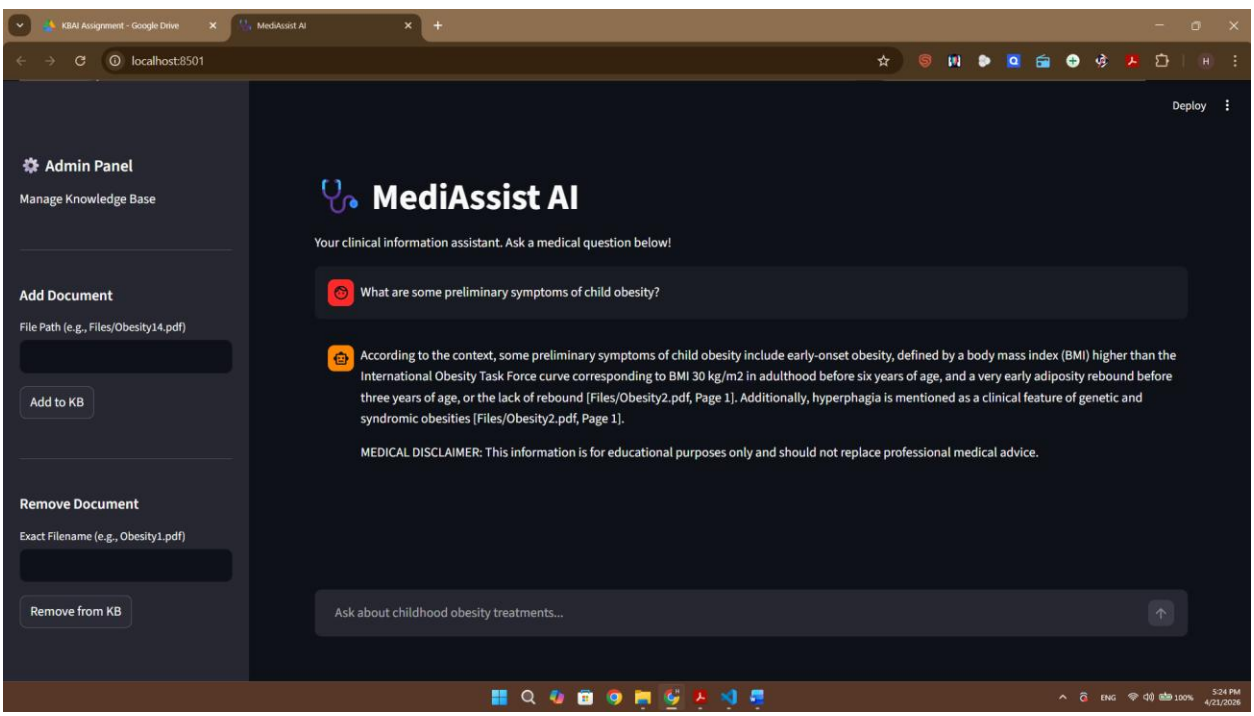
Habiba Abdullah Said Hammad
2203167
KBAI- Assignment 1

```python
@api.post("/chat", response_model=ChatResponse)
async def chat_endpoint(request: ChatRequest):
    try:
        print(f"\n[API] Received question: '{request.question}'")
        result = langgraph_agent.invoke({"question": request.question})
        return ChatResponse(answer=result["answer"])
    except Exception as e:
        print(f"[API] Error: {str(e)}")
        raise HTTPException(status_code=500, detail="Internal server error.")

@api.post("/admin/add_doc")
async def add_doc_endpoint(request: AddDocRequest):
    try:
        print(f"\n[API-ADMIN] Request to add: {request.file_paths}")
        add_documents_to_kb(request.file_paths)
        return {"status": "success", "message": f"Successfully added {request.file_paths} to Knowledge
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

@api.post("/admin/remove_doc")
async def remove_doc_endpoint(request: RemoveDocRequest):
    try:
        print(f"\n[API-ADMIN] Request to remove chunks from: {request.filename}")
        remove_document_from_kb(request.filename)
        return {"status": "success", "message": f"Successfully removed {request.filename} from Knowledge
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))

if __name__ == "__main__":
    print("Starting MediAssist API server with Admin controls...")
    uvicorn.run(api, host="127.0.0.1", port=8000)
```



Habiba Abdullah Said Hammad
2203167
KBAI- Assignment 1
You can also find a video attached in the drive link for the whole demo.